

t and tup

Extreme data quality over quantity: AI models built only from data that seven proof systems agree is correct

Treston Cuzzort

September 2026, updated 2026-09-17

The industry bets on quantity. This project bets on proof.

The industry

Trillions of tokens (Llama 3: over 15 trillion).

Cleaned by rules of thumb, deduplication, and AI models judging quality.

Errors are expected; volume is meant to wash them out.

Hundreds of billions of parameters, huge clusters.

This project

Thousands of examples.

Every one proven by seven independent proof systems, its broken twin caught, its tests passed, never a copy.

No judge's opinion; nothing wrong gets in.

Small models built from nothing on an ordinary computer.

The question: per parameter, does proven data beat piles of data?

When a program says it is correct, how would you know?

How you check	What you learn
Run it once	It worked on that input, today.
Test it on some inputs	It worked on the inputs someone thought of.
Prove it for every input	It does what the specification says, for every input there is.

Today the program may have been written by an AI, faster than anyone can review it.

A test checks one input. A proof checks all of them.

A test

```
assert max(3, 5) == 5
```

One input. It passed, on that input.

A specification, then a proof

```
for all x, y:  
  max(x, y) >= x  
  max(x, y) >= y  
  max(x, y) is x or y
```

Every input. Checked by reasoning, not by running.

Tools that check such proofs: Dafny, Verus, SPARK, Frama-C, Lean 4, Rocq, F*. Each has its own language and its own rules.

The parts, and why each one exists

Part	What it is	Why it exists
seven verifiers	Dafny, Verus, SPARK, Frama-C, Lean, Rocq, F*	any one checker can be wrong; a wrong verdict has to fool all seven
twins	a deliberately broken copy of each program	a proof of an empty promise proves nothing; the promise must catch the twin
t	one small language translated into all seven	write a program once, get seven independent verdicts
lifter and sweeps	Dafny programs translated into t, reggraded in bulk	to test t on programs nobody wrote for it
locallm	builds AI models from random numbers, on any computer	a model whose training data we control completely
the filter loop	models write, t keeps what is proven, models rebuild	the point: extreme quality over quantity, models learn only from what is proven
tup, receipts, lab app	a Linux with build receipts; a live view of every check	so anyone can rerun and watch every claim

Why one verifier is not enough

How it fails	What happened	What the tool said
Mistranslation	You wrote the wrong program into the tool's language.	proved (the wrong program)
An empty specification	The statement is so weak that any program satisfies it.	proved (nothing)
The quiet give-up	The tool timed out or gave up, and the script around it reported a pass.	nothing, and the script said proved

None of the three is the verifier being wrong. All three are invisible to the one tool that produced the verdict.

One small program, seven independent verdicts

```
t 0
task max(x: int, y: int) returns (r: int)
  ensures r >= x
  ensures r >= y
  ensures r == x or r == y
{
  if x >= y {
    r := x;
  } else {
    r := y;
  }
}
```

the signature
the promises

the body

One file, translated by a
script into

Dafny
Verus
SPARK
Frama-C
Lean 4
Rocq
F*

t itself proves nothing.

t/tasks/max.t

Every task gets a twin, a deliberately broken copy

the real program

```
task max(x: int, y: int) returns (r: int)
  ensures r >= x
  ensures r >= y
  ensures r == x or r == y
{
  if x >= y { r := x; }
  else     { r := y; }
}
```

verified

the twin, same specification, broken body

```
task max(x: int, y: int) returns (r: int)
  ensures r >= x
  ensures r >= y
  ensures r == x or r == y
{
  r := x;
}
```

refuted witness x = 0, y = 1: real 1, twin 0

A task counts only when the real program is verified and the twin is refuted at a concrete input. The verifier checks a proof that the promise fails at the witness. A failure to prove is never counted as a refutation.

The seven verifiers

Verifier	Version	Built on
Dafny	4.11.0	.NET, Z3 solver
Verus	0.2026.08.30	Rust, Z3 solver
SPARK (GNATprove)	FSF 16.1.0	Ada, Why3, Z3 solver
Frama-C	33.0	C with ACSL, Alt-Ergo solver
Lean 4	4.33.1	kernel-checked proof terms
Rocq	9.2	kernel-checked proof terms
F*	2026.08.30	Z3 solver

All seven run without root on Linux and macOS; five natively on Windows, all seven under WSL2.

What agreement looks like

task	dafny	verus	spark	framac	lean	rocq	fstar
abs	v / r	v / r	v / r	v / r	v / r	v / r	v / r
max	v / r	v / r	v / r	v / r	v / r	v / r	v / r
gcd	v / r	v / r	v / r	v / r	v / r	v / r	v / r
is_prime	v / r	v / r	v / r	v / r	v / r	v / r	v / r
min_max	v / r	v / r	v / r	v / r	v / r	timeout / refuted	v / r
split_join	v / r	v / r	timeout / refuted	abstain / abstain	v / r	v / r	unproved / refuted

Six of the 34 rows. Each cell reads real / twin; v / r is verified / refuted.

t/AGREEMENT.md, t/CONFORMANCE.md, 2026-09-15

30 of 34

hand-written tasks verified with
the twin refuted in all seven
verifiers

457 of 462

conformance cells: probes with a
declared expected verdict per
verifier

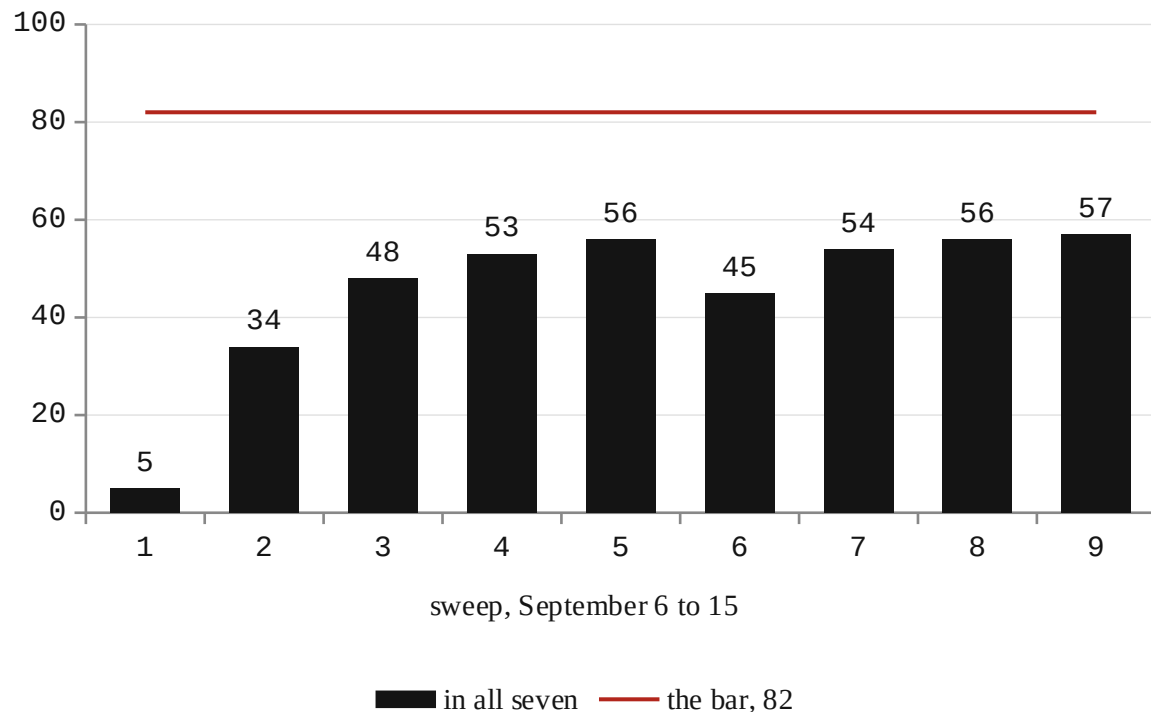
Programs nobody wrote for t



Every translation is checked twice: by a lemma the verifier proves, and by running both programs on the same inputs.

The 35 decisions the lifter makes are recorded with their reasons. A program it cannot translate is refused with the missing construct named.

57 of 164, and the bar is 82



57 of 164

model-written programs
verified with the twin refuted in
all seven, today

82

the bar for 1.0, with a second
person reproducing the table

99 of 164 lift into t
t/COVERAGE-mbpp-dfy-lifter.md

Why the count went down once, on purpose

Before September 12	Since September 12
In five of seven verifiers, a twin the verifier could not prove was counted as refuted. A timeout counted as evidence that the program was wrong.	One door to refuted, in every verifier: it must accept a proof that the specification fails at the witness input. A failure to prove is recorded as unproved and counts for nothing.

56 fell to **45** and was earned back to **57**

with every refutation meaning the same thing.

Auditing the instrument itself

1, 009	randomly generated t tasks, each with its twin
7, 063	verdict cells across the seven verifiers
0	errors shared by all seven
9.2%	of generated twins behaved identically to their real program, so every twin now carries a witness input before any verifier runs

Two holes the audits found and fixed: a definedness gap in the Frama-C translation, and the refuted shortcut. A verifier that proves a twin the interpreter refutes is flagged unsound; that has not happened on the committed tasks.

The corpus t is aimed at

Source	Problems
MBPP	974
HumanEval	164
APPS	10,000
CodeContests	13,610
Total	24,748

nl/manifest.json

772 of 4,239

function-shaped problems inside t's language today (18.2%)

The census ranks what to add next by problems unblocked:

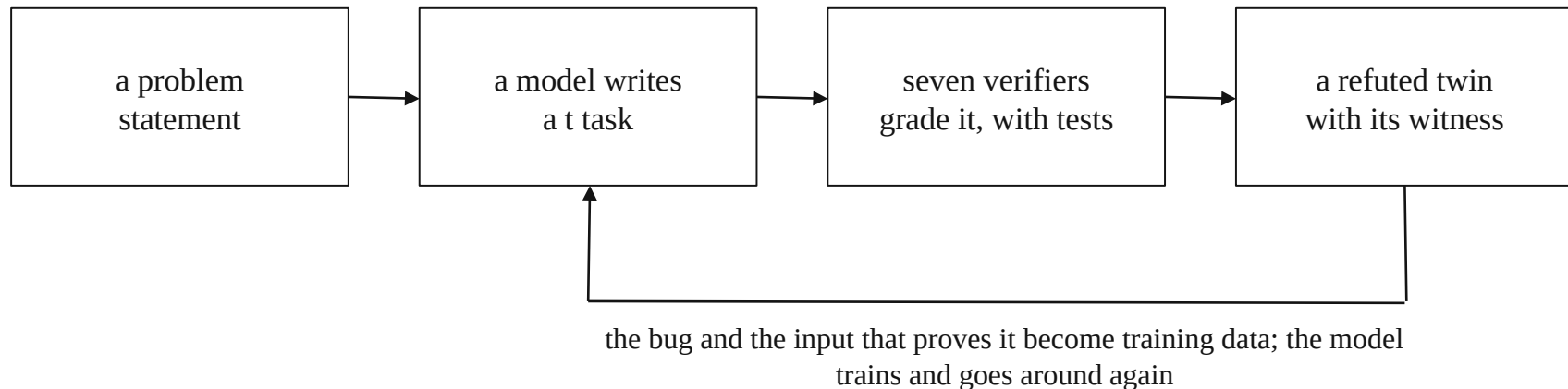
unbounded loops (while true, continue)

library imports beyond math, sys and typing

tuples of three or more, nested tuples

sequences of sequences the reader cannot classify

The training loop



The reward is a specification proved for every input, with a twin proving the specification has content. Tests are checked too. The grader is one script and reproduces the agreement table cell for cell.

What the loop measured with a small model

held-out, 161 problems	before	after two rounds
verified with a refuted twin, some verifier	9	15
same, three samples per problem	12	16
passes the problem's tests	did not move	did not move
replies that fail to parse as t	61%	did not move

Qwen 2.5 Coder, 1.5B parameters, one lab GPU. t/LOOP-CURVE.md

The parse wall. A small model cannot read the parser's error, so it cannot fix it. What changed since: a 27B model on the lab GPUs writes valid t often enough (143 of 368 well-formed, 44 all seven with tests), so the loop now runs with a big model generating and t filtering.

tup is a Linux built from source, with a receipt for every step

```
page: ch08/glibc
command bytes: the book's own <pre> block, verbatim
source: glibc-2.4x.tar.xz   sha256 8a3f...   checked before use
duration: 1h 12m   exit: 0   log sha256: 4c9e...
deviations: overrides/ch08/glibc.sh (states what it changes and why)
```

the shape of one receipt; the real ones are in tup/receipts/

tup 0.1 boots

From its own disk to a login prompt under QEMU, firmware and disk only, in 20 to 45 seconds. Witnessed on macOS, Ubuntu and Windows by someone other than the author.

Witnessed, not verified

It records what was built, from which bytes, in what order, with what outcome. It proves nothing about the kernel or libc. That distinction is not softened anywhere.

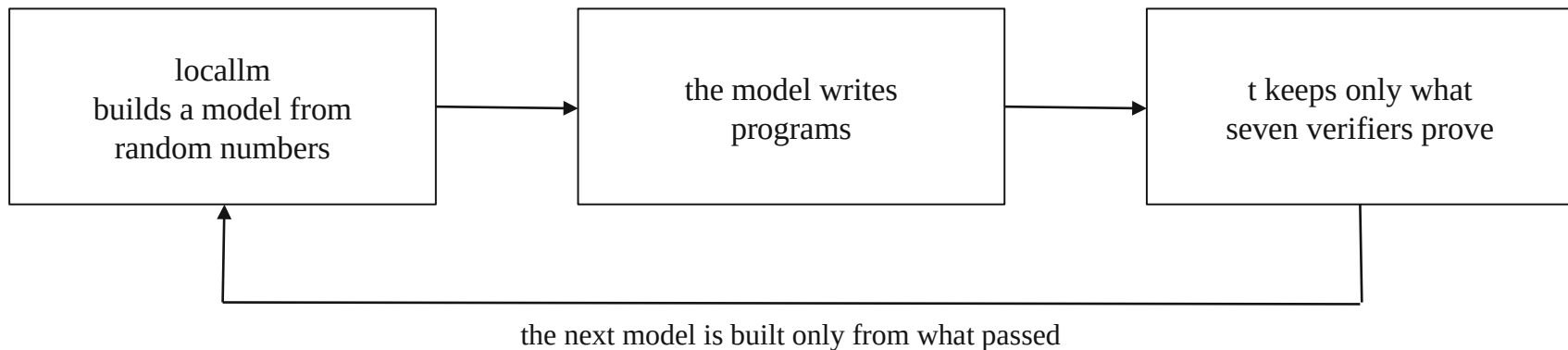
Why the machine is part of the project

+0.55 pp	the trained model against its baseline, $p = .51$: a null
+8.8 to +10.9 pp	the same code and data retrained on a second machine, three times
48 to 58%	of that gain traced to the verifier's Python version

What changed: the verifier ran under Python 3.11.9 on one machine and 3.12.10 on the other. One fails a function that annotates types without importing typing; the other passes it. The retrained models learned to emit that one import line, which appears in none of the 918 training pairs.

So: seven verifiers instead of one, and a machine whose every build step has a receipt.

locallm builds the models; t decides what they learn from



Why build from scratch: a downloaded model has already learned from unfiltered data. A model locallm builds knows only what we gave it, so filtering the data filters the model.

Seven verdicts beside the code

max.t

```
task max(x: int, y: int) returns (r: int)
  ensures r >= x
  ensures r >= y
  ensures r == x or r == y
{
  if x >= y { r := x; }
  else     { r := y; }
}
```

twin: collapse-if, witness x = 0, y = 1

verifier	real	twin
dafny	verified	refuted
verus	absent	absent
spark	verified	refuted
framac	verified	refuted
lean	verified	refuted
rocq	verified	refuted
fstar	verified	refuted

Built: a language server, the VS Code extension, a verdict cache, the walk-through. Not yet run by a second person. A Visual Studio version is planned.

The bar for 1.0, and what blocks it

	The bar	Today
Coverage	82 of the 164 model-written programs read all seven, and a second person reproduces the table from a clean checkout.	57 of 164 in all seven; 99 lift; no second person yet
Usability	A fresh checkout opens a .t file in VS Code on Linux and Windows, and in Visual Studio, with the seven verdicts; a second person repeats the walk-through.	built for VS Code; no second person yet; Visual Studio not built

What blocks the coverage half, by name:

a membership invariant over a growing sequence that Verus, Lean and F* do not yet prove; a Frama-C size bound that must come from a loop invariant; a Rocq bound lemma for recursive specification functions; then lifter rules for function contracts, real numbers, sets and the remaining array shapes.

Eight weeks, 785 commits

Jul 25	first commit: the training pipeline with execution-verified pairs, and locallm
Jul 28	the verifier is pinned into every dataset receipt
Aug	a preregistered null; a replication that did not reproduce; the paper
Aug 31	t goes live with six verifiers; tup 0.1 boots on three hosts; the repository goes public
Sep 2	the pivot: instrument first; the refuted purge
Sep 4 to 11	nine language features, each in all seven verifiers; the lifter
Sep 12 to 15	nine sweeps, 5 to 57 of 164; the loop measured at 1.5B
Sep 15 to 16	a 27B model writes t; the filter loop runs: locallm builds, t filters, locallm rebuilds
Sep 16 to 17	seven verifiers reproduce on a MacBook; a copy-check bug found there and corrected

Every date is a commit. Every number in this talk is in a file that names the commit that produced it.

Quality beat quantity: same model, filtered against raw

each model wrote 500 programs	raw, 925 KB (10x)	raw, 95 KB	t-filtered, 95 KB
parse as t	19	97	375
follow t's rules	4	7	180
clean: new, all seven, twin caught	0	1	29

3.2M parameters, same seeds, same 1,500 steps. t/runs/2026-09-16, recounted in t/runs/2026-09-17

Round 1: rebuild from the clean pool, write again: 31 clean. Rerun on a MacBook: 25.

Corrected in public: first recorded as 46 against 3. The MacBook rerun exposed a copy check that let exact copies count as new. Recounted: 29 against 1. The direction held; the effect is a third smaller.

What it does not do yet: problems it has never seen

232 held-out problems	tests pass	clean: tests and all seven	proven but wrong
Qwen3.8-27B, open model	81	12	7
model locallm built from the clean pool	0	0	188

t/score_heldout.py over t/runs/2026-09-16/loop-data/split-v3.json

Why 0: the clean pool holds only 47 worked problems. The model recites verified programs it memorized (151 exact copies) instead of solving the problem. Verifiers alone would reward that; tests and the copy check do not.

Next, in order: many answers per problem on a home RTX 4080, graded on the lab's CPUs; a clean pool in the thousands; then a small open model trained on it, a locallm model built from it, and Phi-4-mini, all scored on the same 232.

The opposite of industry, set against its best work

Work	How it decides what is good	What it does differently
Llama 3 (Meta), the scaling default	over 15 trillion tokens; rules of thumb, deduplication, model quality scores	thousands of examples, each proven
Phi (Microsoft), "Textbooks Are All You Need"	a model judges data quality; filtered and synthetic textbook data	proofs instead of a judge: seven verifiers plus tests
DeepSeek-R1, RL with verifiable rewards	rewards from tests on some inputs; models learn to game weak checkers	a promise proven for every input; a twin blocks empty promises
AlphaProof, DeepSeek-Prover	the Lean kernel accepts the proof; math problems	seven independent kernels, ordinary programs
AlphaVerus; Vericoding benchmark	translated tasks, a model's critique; one system per task	one program proven in all seven; translator checked

Behind on: scale, a small language, and a held-out score of 0 today. **The bet:** proven data is worth more per parameter. Same model: filtered 29 proven programs, raw 1, ten times the raw data 0.

Limits, then questions

t is small on purpose: no heap, no floats, no concurrency.

Four translations are not hardened against hostile input; the holes are listed.

tup is witnessed, not verified, and arm64 only today.

The filter works at small scale; on unseen problems it scores 0 so far.

A second machine reproduced the table; a second person has not yet.

Try it

```
cd t
python3 run_par.py --jobs 8
bash reproduce.sh --tests
```

Private while the claims move. Research and education use. Ask me for access.

Quality over quantity, proven. Questions?